Verifier-Gated Tool Use: An Agentic Re-Planning Loop with a QAOA Solver for Flood Shelter Assignment

Anonymous Author(s)
Affiliation withheld for double-blind review

***Abstract*—Agentic systems increasingly call external solvers as tools, and quantum optimizers are being proposed as one such tool. A quantum sampler returns a distribution, not a guaranteed answer, so an agent that dispatches its output directly inherits that uncertainty. We study this problem in a flood-evacuation setting. A loop of monitor, planner, solver and verifier agents re-plans the assignment of evacuation groups to shelters as roads close and shelters fill, using the Quantum Approximate Optimization Algorithm (QAOA) with a constraint-preserving XY mixer as one of its tools. Every plan is verified against the current state before dispatch, with fallbacks to a greedy rule, exhaustive search and finally a human. Over 30 five-event episodes from a flood digital twin (150 re-plans, 12 qubits, exact statevector simulation), dispatching the most frequent QAOA sample without checking it would have sent an invalid plan in 21.3% of re-plans. With the verifier, no invalid plan was dispatched and the plan was optimal in 90.0% of re-plans. When a sixth event left 27 of 30 episodes without any feasible plan, the loop escalated every one of them to a human and never escalated a solvable case, while the unverified policy would have dispatched an invalid plan each time. A greedy rule with an exhaustive fallback reached 92.7% in about a millisecond, so the QAOA tool did not beat the classical one. Reusing QAOA angles from the previous event cut objective evaluations by 67%, and a stability term in the cost reduced how many groups were moved at each re-plan. Classical tools solve these instances exactly, so the contribution is the verification and fallback design for probabilistic tools. We claim no quantum speed-up.**

***Index Terms*—agentic AI, tool use, runtime verification, QAOA, XY mixer, re-planning, plan stability, evacuation, digital twin**

I. Introduction

An agent that plans with tools is only as reliable as the weakest tool it trusts. Language-model agents now decide when to call calculators, search engines and code interpreters [1], [2], and optimisation services are a natural next tool. Quantum optimisers are often proposed for this role in logistics and disaster response [3], [4]. Their output, however, is a sample from a probability distribution. Even a well-trained Quantum Approximate Optimization Algorithm (QAOA) circuit [5] returns infeasible or suboptimal bitstrings with non-zero probability, and the probability depends on the instance.

In flood response, the plan that matters is re-made many times. A road closes, a shelter fills or fails, and the officer needs a new assignment of evacuation groups to shelters. Each re-plan should be valid for the state at that moment, should not move groups who are already travelling without reason, and should reach a human when no valid plan exists. A solver cannot see the plan in force or decide when to involve a person, so these requirements fall on the agent loop that calls it.

We design and evaluate such a loop. Our contributions are:

1) An agentic re-planning loop in which a monitor detects state changes, a planner builds a binary optimisation instance, a solver agent calls a portfolio of tools including QAOA, and a verifier checks every candidate plan against the current state, with a fixed fallback order that ends in human escalation (Fig. 1).
2) A stability term that penalises reassigning groups between re-plans, and an angle memory that warm-starts QAOA from the previous event.
3) An evaluation over 30 five-event episodes from a flood digital twin that compares unverified, verified, portfolio, classical and exhaustive policies on validity, optimality, escalation, plan churn and cost.

All QAOA results come from exact, noiseless classical simulation. We do not claim a quantum speed-up. Exhaustive search is included as the reference because it is exact and fast at this size.

II. Related Work

Tool-using agents. ReAct interleaves reasoning traces with tool calls [1], and Toolformer learns when a call is worth making [2]. Surveys of LLM-based agents identify tool reliability and the lack of verification as open problems [6]. Our agents are deterministic programs, which lets us study the tool-handling logic in isolation from language-model behaviour. Variational quantum algorithms are themselves a hybrid loop in which a classical optimiser repeatedly calls a quantum circuit [7]. Our setting wraps this inner loop in an outer, agentic one, and the two loops have different failure modes: the inner loop can fail to train, while the outer loop must cope with a trained circuit that still returns a bad sample.

Verified fallbacks. The Simplex architecture runs an unverified high-performance controller alongside a verified safety controller and switches when the former leaves a safe envelope [8]. Shielding applies the same idea to learned policies [9]. Our verifier plays the role of the switching logic, with the QAOA tool as the high-performance component and exhaustive search as the verified one. Runtime verification more generally checks each execution of a system against a specification as it happens, rather than proving the system correct in advance [10]. For a probabilistic solver, that is the only kind of guarantee

available, because the same call can return a different answer each time.

Human escalation. Parasuraman *et al.* distinguish levels of automation, from a system that only suggests options to one that acts and informs the operator afterwards, and argue that the level should fall as the cost of an error rises [11]. Our loop acts on its own only when a plan passes the verifier and hands the decision to a person otherwise, so the level of automation drops exactly when the tools cannot produce a valid plan.

Re-planning and stability. In automated planning, repairing a plan rather than re-planning from scratch keeps the new plan close to the old one, which reduces disruption for whoever is executing it [12]. Evacuation shelter allocation has a long history in operations research [13]. Flood-risk professionals who co-designed a digital twin asked above all for transparent provenance and "what-if" runs they could trust [14]. A re-planning loop driven by such a twin has to explain why a plan changed, and a stability term that leaves groups where they are unless moving them is clearly better makes those explanations shorter.

QAOA as a solver. The quantum alternating operator ansatz uses mixers that preserve hard constraints [15], and XY mixers keep one-hot encodings valid [16]. Optimal QAOA angles concentrate across similar instances [17] and can be transferred between them [18], and angles at depth p can be interpolated from depth $p-1$ [19]. Penalty formulations for assignment problems are described by Lucas [20].

Research gap. Studies of tool-using agents assume tools whose answers are deterministic or checked by the language model itself, and QAOA studies measure the quality of a single call. To our knowledge, how an agent should verify, fall back from and reuse a probabilistic optimiser across a sequence of changing problems has not been measured. That is the question this paper addresses.

III. Problem and Agents

A. Assignment Instance

The digital twin models three population zones, nine junctions and three shelters joined by 22 roads, each with a travel time, flood index and congestion level from a fixed seed. Each episode follows the first four evacuation groups (8–22 people each). For group g and shelter s, the cost c_{gs} is the travel time in minutes plus five times the mean flood index of the best open path. A pair is forbidden if no open path exists, the shelter has failed or the group exceeds its remaining seats R_s. With $x_{gs} \in \{0,1\}$, the planner minimises

$$J(\mathbf{x}) = \sum_{g,s} \big(c_{gs} + \mu\,[s \neq s_g^{\mathrm{prev}}]\big) x_{gs} \tag{1}$$

subject to one shelter per group, no forbidden pair and $\sum_g n_g x_{gs} \le R_s$. Here s_g^{prev} is the shelter assigned to group g by the plan in force and μ (minutes) is the stability weight. The term is linear in $\mathbf{x}$, so it adds no qubits or two-qubit gates.

For QAOA the planner writes the problem as a quadratic unconstrained binary optimisation with costs shifted by their per-group minimum, a penalty for forbidden pairs and a pairwise penalty $x_{gs}x_{hs}$ for any two groups that together exceed R_s. With $c'_{gs} = c_{gs} + \mu[s \neq s_g^{\mathrm{prev}}] - \min_{s'}(\cdot)$ the shifted cost, $\mathcal{F}$ the forbidden pairs and $\mathcal{O}$ the triples (g,h,s) with $n_g + n_h > R_s$, the energy is

$$E(\mathbf{x}) = \sum_{g,s} c'_{gs} x_{gs} + P\Big[\sum_{(g,s)\in\mathcal{F}} x_{gs} + \sum_{(g,h,s)\in\mathcal{O}} x_{gs}x_{hs}\Big], \tag{2}$$

where $P = 1 + \sum_g \max_s c'_{gs}$ exceeds the total cost spread, so any penalised bitstring has higher energy than any unpenalised one. The one-hot constraint needs no penalty here because the mixer preserves it. The pairwise encoding can miss an overload by three groups, which is one reason the verifier re-checks every plan exactly. The solver uses a ring-XY mixer within each group, started from a product of W states, so every sample is one-hot by construction [16]. The circuit has depth $p = 3$, its angles are trained with COBYLA on the mean energy, and 64 shots are drawn from the final state.

B. Episode

Each episode replays five events on a seeded twin: E0, the initial state; E1, road R5 closes; E2, Shelter A receives arrivals equal to half its capacity; E3, a further randomly chosen road closes; and E4, Shelter A fails. After each event the monitor compares the graph version and shelter states with those at the last plan. If nothing has changed, no re-plan is made.

C. Agents and Policies

Fig. 1 shows the loop, and Algorithm 1 gives the verified policy. Let σ_t be the twin state after event t, with forbidden pairs $\mathcal{F}(\sigma_t)$ and remaining seats $R_s(\sigma_t)$. The verifier accepts a plan $\mathbf{x}$ when

$$V(\mathbf{x},\sigma_t) = \Big[\textstyle\sum_s x_{gs} = 1\ \forall g\Big] \wedge \Big[x_{gs} = 0\ \forall (g,s) \in \mathcal{F}(\sigma_t)\Big] \wedge \Big[\textstyle\sum_g n_g x_{gs} \le R_s(\sigma_t)\ \forall s\Big]. \tag{3}$$

It reads the current state, not the instance the solver saw, so a plan that became invalid while the solver ran is rejected. Plan churn at event t is the number of groups whose shelter changed,

$$C_t = \sum_g \big[s_g(\mathbf{x}_t) \neq s_g(\mathbf{x}_{t-1})\big], \tag{4}$$

and the portfolio policy keeps $\arg\min\{J(\mathbf{x}) : \mathbf{x} \in \{\mathbf{x}_{\mathrm{QAOA}}, \mathbf{x}_{\mathrm{greedy}}\},\ V(\mathbf{x},\sigma_t)\}$. We compare five policies on the same episodes:

- **Raw**: dispatch the most frequent QAOA sample without checking it.
- **Verified**: take the cheapest feasible QAOA sample; fall back to greedy, then exhaustive search, then a human.
- **Portfolio**: run verified QAOA and the greedy rule and keep the cheaper valid plan.
- **Classical**: greedy, then exhaustive search, then a human.
- **Oracle**: exhaustive search over all 3^G one-hot plans.

The greedy rule assigns the largest group first to its cheapest shelter with seats left.

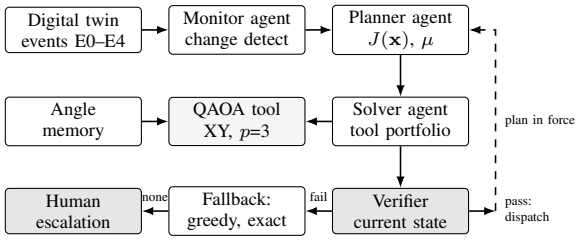


Fig. 1. Agentic re-planning loop. Shaded blocks decide whether a plan may be dispatched. The plan in force feeds the stability term of the next re-plan, and the angle memory warm-starts the QAOA tool.

Algorithm 1 Verified re-planning at one event

if state key unchanged since last plan **then**
  return plan in force
end if
build J from the current state and plan in force
$\boldsymbol{\theta} \leftarrow$ memory, or INTERP training from $p = 1$
draw 64 shots; $\mathbf{x} \leftarrow$ cheapest one-hot sample passing the verifier
if none **then** $\mathbf{x} \leftarrow$ greedy; **if** it fails, $\mathbf{x} \leftarrow$ exhaustive
if $\mathbf{x}$ passes the verifier **then**
  dispatch $\mathbf{x}$; store $\mathbf{x}$ and $\boldsymbol{\theta}$
else
  escalate to a human with the failed checks
end if

Tool costs. The three tools scale very differently. With G groups and S shelters, the greedy rule needs $O(GS)$ operations after sorting, exhaustive search checks S^G plans, and exact statevector simulation of QAOA stores 2^{GS} amplitudes and applies $O(p\,GS)$ gates to them per evaluation. At $G = 4$ and $S = 3$ these are 12 operations, 81 plans and 4,096 amplitudes. Exhaustive search therefore remains the right verifier at this size, and simulation is the bottleneck, not the optimisation. On hardware, the circuit would need GS qubits, so the loop's structure would stay the same while the solver tool changed; that is the setting in which the verifier and fallback order matter most, because exhaustive search would no longer be available as the last resort. To isolate the effect of verification, the raw, verified and portfolio policies are scored on the same QAOA draw at each event, which is computed on the instance of the verified policy.

Implementation. The agents are plain Python functions that share one state object. The monitor hashes the graph version and every shelter's status and remaining seats into a state key and compares it with the key stored at the last plan. The planner reads the twin through the same functions that the routing benchmark uses, so the instance and the verifier always see the same road and shelter data. The solver agent calls each tool through one interface that returns either a plan or nothing, and records the tool's running time and evaluation count. The verifier is a pure function of the plan and the current state, which makes it easy to test exhaustively on small instances.

IV. Experimental Setup

We ran 30 episodes with twin seeds $42+17i$, $i = 0, \ldots, 29$, giving 150 events. Every event changed the state, so the monitor triggered a re-plan at all 150 events. At least one feasible plan existed at every event, so the escalation branch of the loop was not reached in these episodes. To test it, we extended each episode with a sixth event, E5, in which Shelter B also fails and only Shelter C remains, and ran the verified policy with angle memory through all six events. Each QAOA call was simulated as an exact 2^{12}-amplitude statevector. In the baseline configuration ($\mu = 0$, cold start), each call trained depths 1 to 3 with INTERP initialisation [19] and two random restarts per depth. With angle memory, a call started a single optimisation from the angles stored at the previous event. We also ran the verified and oracle policies with $\mu = 3$ and $\mu = 6$ minutes, with memory on. Times are wall-clock on a consumer laptop and include simulation, so they describe this prototype, not a quantum device.

TABLE I
Re-planning policies, 150 re-plans ($\mu = 0$, cold start)

Policy	Invalid (%)	Esc.	Opt. (%)	Gap (min)	ms
QAOA top sample, unverified	21.3	0	28.7	1.48	8143.3
QAOA + verifier + fallback	0.0	0	90.0	0.14	8143.2
Portfolio (QAOA ‖ greedy)	0.0	0	95.3	0.02	8143.3
Greedy + exact fallback	0.0	0	92.7	0.20	1.1
Exhaustive (oracle)	0.0	0	100.0	0.00	1.0

Invalid = dispatched plan violating a constraint. Opt. = optimal plans over all re-plans (an invalid plan is never optimal); Gap over valid plans.

TABLE II
Re-plans, infeasible events and unverified failures by event

Event	Re-plans	No feasible plan	Raw invalid
E0 initial	30	0	0
E1 road R5 closed	30	0	0
E2 Shelter A surge	30	0	9
E3 extra closure	30	0	10
E4 Shelter A failed	30	0	13

V. Results

A. Verification and Fallback

Table I summarises the baseline configuration. Dispatching the most frequent QAOA sample without checking would have sent an invalid plan in 21.3% of re-plans. The XY mixer guarantees one-hot samples, so these failures were capacity and forbidden-pair violations. They were concentrated in the events that made capacity tight (Table II). The verified policy dispatched no invalid plan and reached the exhaustive optimum in 90.0% of re-plans, with a mean excess cost of 0.14 min. In 2.7% of re-plans none of the 64 shots was feasible and the greedy fallback supplied the plan.

The portfolio policy reached 95.3% optimal plans; the greedy rule supplied the kept plan in 8.7% of re-plans. The classical policy reached 92.7%, more than verified QAOA, at a fraction of the time. Its mean time per re-plan was 1.1 ms, against 8143.2 ms of simulated training and sampling for the verified QAOA policy.

TABLE III
OPTIMAL PLANS (%) AND FALLBACKS BY EVENT ($\mu = 0$, COLD START, 30 EPISODES)

Event	Raw	Verified	Portfolio	Classical	Fallbacks
E0 initial	16.7	100.0	100.0	100.0	0
E1 road R5 closed	40.0	96.7	100.0	100.0	0
E2 Shelter A surge	23.3	80.0	93.3	93.3	2
E3 extra closure	26.7	86.7	90.0	83.3	1
E4 Shelter A failed	36.7	86.7	93.3	86.7	1

Raw counts only valid plans as optimal; an invalid raw plan is never optimal.

TABLE IV
ONE EPISODE IN FULL (SEED 59, $\mu = 0$, COLD START)

Event	Raw	Verified tool	Verified	Churn	Opt. cost
E0 initial	optimal	QAOA	optimal	0	75.94
E1 R5 closed	+4.75	QAOA	optimal	0	75.94
E2 surge	+0.64	QAOA	optimal	0	75.94
E3 closure	invalid	QAOA	optimal	0	75.94
E4 A failed	invalid	QAOA	optimal	2	88.56

+x = excess cost in minutes over the optimum; fb. = fallback.

Table III breaks the cold configuration down by event. All 4 fallbacks happened after E1, when the surge, the extra closure or the shelter failure had removed many plans from the feasible set. With N independent shots and a feasibility probability P_{feas}, the chance that no shot is feasible is

$$P_{\text{fb}} = (1 - P_{\text{feas}})^N, \tag{5}$$

which is negligible for $N = 64$ unless P_{feas} falls to a few percent. The observed fallbacks therefore point to events where the trained state put almost no weight on feasible plans, not to an unlucky draw.

B. A Worked Episode

Table IV follows one episode in full, seed 59, the first in which the raw policy would have dispatched an invalid plan. The unverified top sample was invalid at E3 closure, E4 A failed. At each of these events the verifier rejected it, and the best feasible sample was dispatched instead. The churn column shows how many groups the verified plan moved relative to the plan already in force, and the last column gives the travel cost of the exhaustive optimum for reference. Fig. 2 summarises the same information over all 30 episodes: the verified and portfolio policies dispatched an optimal plan in at least 80% and 90% of re-plans at every event, while the raw policy never exceeded 40%.

C. Escalation

With Shelter B failed as well as Shelter A, the remaining shelter could not take all four groups in 27 of the 30 episodes. In each of these the QAOA tool returned no feasible sample, the greedy fallback failed and exhaustive search confirmed that no plan existed, so the loop escalated to a human with the list of failed checks. There were 0 wrong escalations, cases escalated although a valid plan existed, and 0 missed ones. The raw policy, which dispatches the most frequent sample, would have sent an invalid plan in all 27 infeasible cases, since it has no way to recognise that no valid plan exists. This is the situation in which an unverified tool is most dangerous: every answer it can give is wrong, and it still gives one.

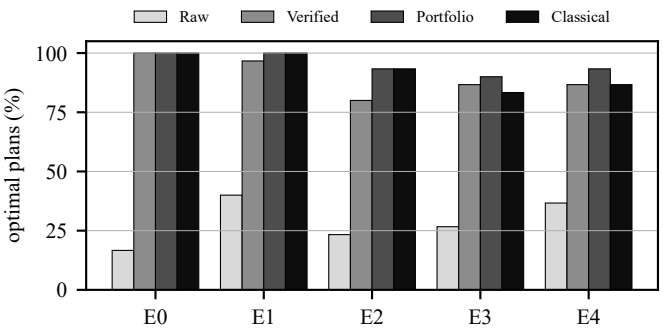


Fig. 2. Share of re-plans in which each policy dispatched an optimal plan, by event ($\mu = 0$, cold start, 30 episodes).

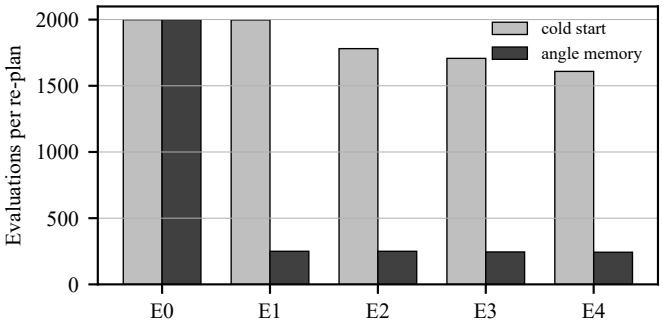


Fig. 3. Mean QAOA objective evaluations per re-plan by event, without and with angle memory ($\mu = 0$).

D. Angle Memory

Starting each QAOA call from the previous event's angles reduced the mean number of objective evaluations from 1819 to 597 (67% fewer) and the mean time per verified re-plan from 8143.2 to 2800.1 ms. The verified policy reached the optimum in 92.0% of feasible re-plans with memory, against 90.0% without. Consecutive events change only a few costs and capacities, so the optimal angles move little. This agrees with reports that QAOA angles concentrate across similar instances [17], [18]. Fig. 3 shows where the saving came from. With memory, the first re-plan of each episode (E0) still paid the full cost of depth-progressive training, 1999 evaluations on average, and every later event needed only 247. In a long incident with many re-plans, the cost per re-plan would approach the later figure.

E. Robustness Across Configurations

Table V repeats the main comparison in all four configurations. The verified and portfolio policies dispatched no invalid plan in any of them. The share of invalid plans that the raw policy would have dispatched fell from 21.3% with cold starts to 14.7% with angle memory and 6.7% with $\mu = 6$. We did

TABLE V
POLICIES ACROSS CONFIGURATIONS (150 RE-PLANS EACH)

μ	angles	Raw invalid	Ver. opt.	Port. opt.	Class. opt.	Evals
0	cold	21.3	90.0	95.3	92.7	1819
0	memory	14.7	92.0	95.3	92.7	597
3	memory	12.0	98.7	100.0	96.7	598
6	memory	6.7	98.7	100.0	98.7	598

All values in %, except Evals = mean QAOA objective evaluations per re-plan.

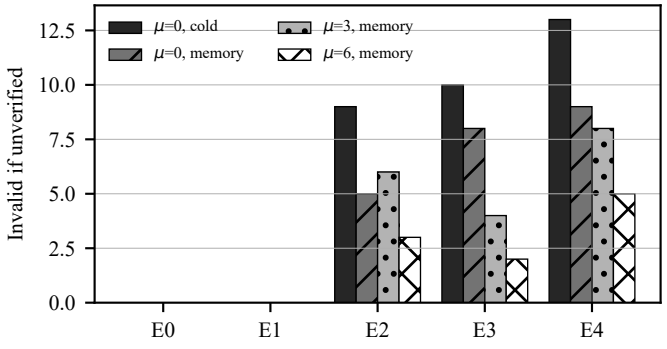


Fig. 4. Episodes (out of 30) in which the most frequent QAOA sample would have been an invalid plan, by event and configuration.

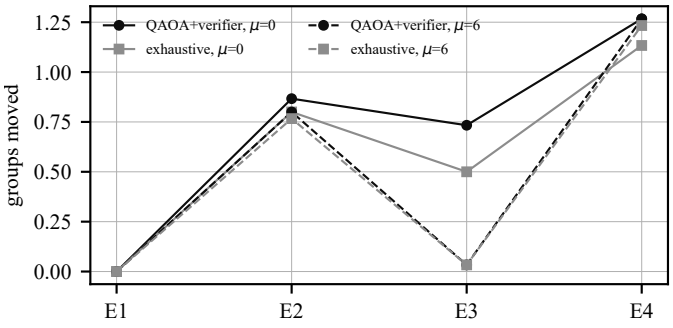


Fig. 5. Groups reassigned per re-plan at each event, for the verified QAOA policy and exhaustive search, with and without the stability term (angle memory on).

TABLE VI
STABILITY WEIGHT μ (MINUTES), WITH ANGLE MEMORY

	QAOA + verifier		Exhaustive	
μ	churn	cost (min)	churn	cost (min)
0	0.72	80.73	0.61	80.60
3	0.57	80.73	0.53	80.78
6	0.53	80.97	0.51	80.92

churn = groups reassigned per re-plan after E0;
cost = mean travel cost of dispatched plans.

not isolate the cause. A plausible reason is that warm-started angles are already tuned to a closely related instance and that the stability term raises the energy of plans that move groups, so both concentrate probability on plans near one that was valid a moment ago. The raw policy is still unsafe in every configuration. Fig. 4 shows that its failures occurred only after the shelter surge, the extra closure and the shelter failure: 9, 10 and 13 of 30 episodes in the cold configuration, and none at E0 or E1 in any configuration. These are the events where the cheapest-looking plan overloads a shelter.

F. Plan Stability

Table VI and Fig. 6 show the effect of the stability weight. For the exhaustive planner, raising μ from 0 to 6 minutes changed the mean number of groups reassigned per re-plan from 0.61 to 0.51, while the mean travel cost of the dispatched plan changed from 80.60 to 80.92 min. The verified QAOA policy followed the same direction. Without the stability term it reassigned 0.82 groups per re-plan, against 0.61 for exhaustive search, because QAOA samples choose among plans of nearly equal cost at random. A small μ breaks these ties in favour of the plan in force, and at $\mu = 6$ the verified policy reached the optimum of (1) in 98.7% of re-plans. Fig. 5 shows where the churn occurred. The largest single share came with E4, when Shelter A failed and its groups had to move whatever the weight: the verified policy moved 1.27 groups per episode at E4 with $\mu = 0$ and the same 1.27 with $\mu = 6$. The stability term instead removed almost all reassignments at E3, from 0.73 to 0.03 groups per episode.

A small μ removes reassignments between plans of nearly equal cost, which an officer would otherwise have to communicate to teams already on the road.

Reproducibility. The episodes, events and QAOA draws are fixed by the seeds above, and the simulator, the agent loop and the tables are generated by scripts in the prototype repository. Wall-clock times are the only values that change between runs.

VI. DISCUSSION AND LIMITATIONS

The verifier did the work that made the QAOA tool usable. Without it, a solver that was right most of the time still produced invalid dispatches, and these failures clustered in the events where capacity was tight, which are the events that matter most. With it, the loop never dispatched an invalid plan. In the extended episodes it handed every infeasible situation to a human, and it never escalated a situation that had a valid plan. The same structure applies to any probabilistic tool, whether a quantum sampler, a heuristic or a learned policy. We draw four design rules from the experiments. First, verify against the state at dispatch time, not the state the tool saw. Second, prefer the best verified sample to the most frequent sample, since the most frequent sample was invalid in up to a fifth of re-plans. Third, run a cheap classical tool in parallel and keep the better valid plan, which raised optimality at no risk. Fourth, carry tool state such as QAOA angles across events, because consecutive re-planning problems are similar and the saving in evaluations was large.

Latency also matters in practice. The verified QAOA policy needed seconds of simulated training per re-plan, and a hardware run would add queueing and many circuit executions per evaluation. An evacuation plan needed within a minute could not wait for such a tool without a classical plan already in hand. The portfolio design handles this: the greedy plan is

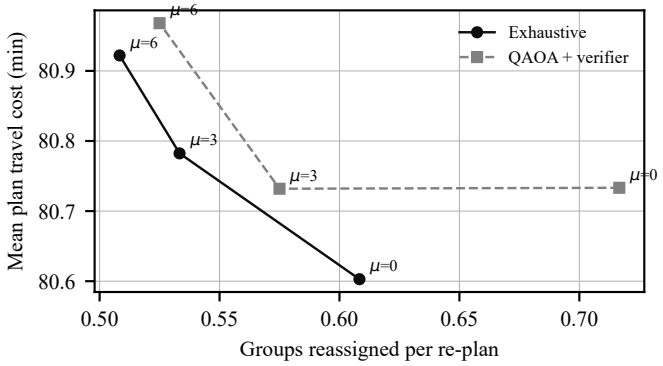


Fig. 6. Plan churn against travel cost for three stability weights.

available immediately and the QAOA plan replaces it only if it arrives in time and is better.

The results do not support using QAOA on these instances in practice. Exhaustive search over 81 plans is exact and takes milliseconds, and the greedy rule is nearly as good. The QAOA tool earns its place in the loop only as a stand-in for future solvers on instances too large to enumerate, and our experiments say nothing about how it would perform on hardware or at that scale.

Several threats to validity apply. The raw, verified and portfolio policies share one draw per event, so their differences come from the selection rule alone, but the run-to-run variance of a single draw is not measured. The greedy rule and exhaustive search are deterministic and see the same instances, which favours them in a comparison of optimality. The optimality figures are relative to (1) with the current μ, so a plan that is optimal at $\mu = 6$ can cost more travel time than the $\mu = 0$ optimum. Further limits apply. All circuits were noiseless simulations at 12 qubits. Each episode follows one fixed sequence of events on a small synthetic network, and accessibility never constrained the plan because every shelter is accessible. The stability weight is a single scalar that treats all reassignments as equal, whereas moving a group already on the road costs more than moving one that has not left. The raw, verified and portfolio policies shared one QAOA draw per event, which makes them directly comparable but does not show the variance between draws. Escalation was tested with one kind of infeasibility, a double shelter failure, and no officer took part in the evaluation.

VII. Conclusion

We built an agentic loop that uses QAOA as one tool among several for re-planning flood shelter assignments, and placed a verifier and a fixed fallback order between the tool and dispatch. On 30 digital-twin episodes, unverified QAOA output would have produced invalid plans in 21.3% of re-plans. The verified loop produced none, escalated all 27 infeasible situations created by a double shelter failure, and never escalated a solvable one, although a greedy rule reached the optimum more often than QAOA at a tiny fraction of the cost. Angle memory and a stability term made re-planning cheaper and less disruptive. For agent design, the results mean that a probabilistic tool needs a verifier that checks its output against the current state, a fallback, and a route to a human. They make no case for quantum advantage.

Acknowledgment

The authors thank the OpenStreetMap contributors and Open-Meteo for openly available data used by the digital twin. In line with the IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic) was used to help edit and restructure the text of all sections and to draft the simulation, experiment and plotting scripts. All experiments use the authors' own software and can be reproduced with fixed seeds. The authors checked every number, claim and reference and take full responsibility for the content.

References

[1] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[2] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Adv. Neural Inf. Process. Syst. 36*, 2023.

[3] A. Abbas *et al.*, "Challenges and opportunities in quantum optimization," *Nat. Rev. Phys.*, vol. 6, pp. 718–735, 2024.

[4] A. R. Cadeliña *et al.*, "D-Wave implementation of quantum annealing for optimal resource allocation in disaster response operation of Marikina City," *ECTI Trans. Comput. Inf. Technol.*, vol. 18, no. 1, pp. 24–33, 2024.

[5] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[6] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024.

[7] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, "The theory of variational hybrid quantum-classical algorithms," *New J. Phys.*, vol. 18, no. 2, Art. no. 023023, 2016.

[8] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.

[9] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2669–2678.

[10] M. Leucker and C. Schallhart, "A brief account of runtime verification," *J. Log. Algebr. Program.*, vol. 78, no. 5, pp. 293–303, 2009.

[11] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, 2000.

[12] M. Fox, A. Gerevini, D. Long, and I. Serina, "Plan stability: Replanning versus plan repair," in *Proc. 16th Int. Conf. Automated Planning Scheduling (ICAPS)*, 2006, pp. 212–221.

[13] H. D. Sherali, T. B. Carter, and A. G. Hobeika, "A location-allocation model and algorithm for evacuation planning under hurricane/flood conditions," *Transp. Res. B, Methodol.*, vol. 25, no. 6, pp. 439–452, 1991.

[14] H. Underhill, L. McEwen, and T. Coulthard, "'That's the dream, right?': Reflections on the co-design of an environmental digital twin by flood risk management professionals," *Front. Environ. Sci.*, vol. 13, Art. no. 1724250, 2026.

[15] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, Art. no. 34, 2019.

[16] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz," *Phys. Rev. A*, vol. 101, no. 1, Art. no. 012320, 2020.

[17] F. G. S. L. Brandão, M. Broughton, E. Farhi, S. Gutmann, and H. Neven, "For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances," arXiv:1812.04170, 2018.

[18] A. Galda, X. Liu, D. Lykov, Y. Alexeev, and I. Safro, "Transferability of optimal QAOA parameters between random graphs," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2021, pp. 171–180.

[19] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices," *Phys. Rev. X*, vol. 10, no. 2, Art. no. 021067, 2020.
[20] A. Lucas, "Ising formulations of many NP problems," *Front. Phys.*, vol. 2, Art. no. 5, 2014.